Projet C++ : Modèles de Volatilité

Rapport final

Master 2 Mathématiques et Applications (Finance)

26 avril 2026

Table des matières

1 Présentation du problème et des modèles **2**

2 Fondements mathématiques **2**
- 2.1 Surface de volatilité implicite 2
- 2.2 Variance totale implicite et log-moneyness 2
- 2.3 Volatilité locale et formule de Dupire 3
- 2.4 Tests d'arbitrage 3
- 2.5 Modèle de Heston 3

3 Choix d'implémentation C++ **4**
- 3.1 Architecture générale 4
- 3.2 Modules de la partie 1 4
- 3.3 Modules de l'option A 4

4 Résultats numériques et comparaisons **5**
- 4.1 Validation de la question 1.1 5
- 4.2 Visualisation de la surface de volatilité implicite 5
- 4.3 Question 1.2 : volatilité locale 6
- 4.4 Question 1.3 : tests d'arbitrage 6
- 4.5 Comparaison des schémas sous Heston 7

5 Analyse critique et limites **7**

6 Conclusion **7**

1 Présentation du problème et des modèles

L'objectif de ce projet est d'implémenter en C++ une chaîne cohérente de traitement de modèles de volatilité utilisés en finance quantitative. Le travail est organisé en deux parties.

La première partie consiste à partir de données discrètes de marché, sous la forme de volatilités implicites

$$\sigma_{\text{imp}}(T, K),$$

afin de construire une surface continue de volatilité implicite, puis d'en déduire une surface de volatilité locale via la formule de Dupire.

La seconde partie repose sur l'étude d'un modèle avancé. Dans ce projet, l'option retenue est l'**option A**, consacrée au **modèle de Heston**. L'objectif est alors de comparer plusieurs méthodes numériques de simulation de la variance et du sous-jacent, puis de confronter les prix Monte Carlo obtenus à un prix semi-analytique de référence.

L'intérêt du projet est double :

— sur le plan financier, il relie directement la surface de volatilité implicite observée sur le marché à des modèles de diffusion plus avancés ;

— sur le plan informatique, il impose de structurer le code de manière modulaire, orientée objet, stable numériquement et testable.

La logique générale suivie est la suivante :

données de marché $\longrightarrow$ surface implicite $\longrightarrow$ volatilité locale $\longrightarrow$ modèle de Heston $\longrightarrow$ simulation $\longrightarrow$ pricing

2 Fondements mathématiques

2.1 Surface de volatilité implicite

Les données de départ sont des volatilités implicites de marché observées sur une grille discrète de maturités T et de strikes K. Le premier objectif est de construire une fonction continue

$$(T, K) \mapsto \sigma_{\text{imp}}(T, K),$$

à partir de ces points discrets.

Dans le cadre de ce projet, une interpolation bilinéaire a été retenue. Ce choix est simple, robuste et suffisant pour obtenir une première surface continue exploitable sur tout le domaine couvert par les données.

2.2 Variance totale implicite et log-moneyness

Pour la construction de la volatilité locale, on introduit la **variance totale implicite**

$$w(K, T) = \widetilde{\sigma}^2(K, T)\, T,$$

où $\widetilde{\sigma}(K, T)$ désigne la volatilité implicite Black–Scholes.

On introduit également la variable de **log-moneyness**

$$y = \ln\left(\frac{K}{F_T}\right), \qquad F_T = S_0 e^{rT},$$

où F_T est le forward à maturité T, S_0 le spot initial et r le taux sans risque.

Ainsi, la surface de variance totale est considérée comme une fonction

$$w = w(y, T).$$

2.3 Volatilité locale et formule de Dupire

La variance locale est obtenue à partir de la formule de Dupire exprimée en fonction de la variance totale implicite. On écrit

$$\sigma_{LV}(y,T) = \sqrt{v_L(y,T)},$$

où $v_L(y,T)$ dépend de $\partial_T w$, $\partial_y w$ et $\partial^2_{yy} w$.

Dans l'implémentation, ces dérivées sont approchées numériquement par différences finies centrées :

$$\frac{\partial w}{\partial T}(y,T) \approx \frac{w(y,T+\Delta T) - w(y,T-\Delta T)}{2\Delta T},$$

$$\frac{\partial w}{\partial y}(y,T) \approx \frac{w(y+h,T) - w(y-h,T)}{2h},$$

$$\frac{\partial^2 w}{\partial y^2}(y,T) \approx \frac{w(y+h,T) - 2w(y,T) + w(y-h,T)}{h^2}.$$

L'intérêt du passage à w et y est d'améliorer la stabilité numérique du calcul.

2.4 Tests d'arbitrage

Une surface de volatilité ou de prix n'est exploitable que si elle respecte des propriétés élémentaires de non-arbitrage. Les tests considérés sont :

— la monotonie du prix en strike ;

— la convexité du prix en strike ;

— l'absence d'arbitrage calendaire.

Ces tests constituent une validation financière minimale de la surface construite.

2.5 Modèle de Heston

Pour la seconde partie, on considère le modèle de Heston sous la mesure risque-neutre :

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^S,$$

$$dv_t = \kappa(\theta - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^v, \qquad dW_t^S\,dW_t^v = \rho\,dt.$$

Le processus v_t suit donc une dynamique de type CIR. Ce modèle est particulièrement intéressant car il introduit une volatilité stochastique tout en conservant une structure suffisamment riche pour reproduire des sourires de volatilité réalistes.

Dans ce projet, plusieurs schémas numériques sont comparés :

— Euler ;

— Truncated Gaussian (TG) ;

— Quadratic Exponential (QE) ;

— Broadie–Kaya simplifié ;

— formule semi-analytique de Heston comme benchmark.

3 Choix d'implémentation C++

3.1 Architecture générale

Le code a été conçu de manière modulaire, avec séparation entre fichiers d'interface `.h` et fichiers d'implémentation `.cpp`. L'arborescence générale suit la logique suivante :

- `include/` : interfaces et définitions des classes ;
- `src/` : implémentations ;
- `data/` : données d'entrée ;
- `build/` ou `cmake-build-debug/` : fichiers générés à la compilation.

La compilation a été gérée par `CMake`, ce qui permet une organisation claire du projet et une reconstruction reproductible.

3.2 Modules de la partie 1

Module `DataLoader` Ce module lit le fichier CSV, parse les colonnes utiles, convertit les valeurs au bon type et stocke les points de marché.

Module `ImpliedVolGrid` Ce module construit la grille discrète en maturité et en strike, trie les axes, range les volatilités au bon endroit et vérifie la cohérence globale de la grille.

Module `BilinearInterpolator` Ce module implémente l'interpolation bilinéaire. Il repère les quatre coins autour d'un point (T, K), effectue les interpolations successives en strike et en maturité, et gère les cas de bord simples.

Module `ImpliedVolSurface` Ce module centralise l'évaluation de la fonction continue $\sigma_{\text{imp}}(T, K)$. Il s'appuie sur la grille et l'interpolateur.

Module `LocalVolSurface` Ce module calcule la variance totale implicite, les dérivées numériques et la volatilité locale via Dupire.

Module `main/tests` Le programme principal et les fichiers de test servent à construire la chaîne complète, exécuter les vérifications numériques et afficher les résultats intermédiaires.

3.3 Modules de l'option A

L'option A a été organisée en trois niveaux.

Niveau aléatoire

- `NormalRng.cpp` : génération de gaussiennes standards ;
- `CorrelatedRng.cpp` : construction de gaussiennes corrélées.

Niveau dynamique

- `CIRProcess.cpp` : mise à jour de la variance ;
- `HestonProcess.cpp` : dynamique jointe du spot et de la variance.

Niveau pricing et comparaison

- `EulerSchemePricer.cpp` ;
- `TruncatedGaussianSchemePricer.cpp` ;
- `QuadraticExponentialSchemePricer.cpp` ;
- `BroadieKayaPricer.cpp` ;
- `HestonSemiAnalyticalPricer.cpp` ;
- `test_option_a.cpp`.

Cette architecture permet de séparer proprement :

1. la source d'aléa ;
2. la dynamique du modèle ;
3. la valorisation et la comparaison des méthodes.

4 Résultats numériques et comparaisons

4.1 Validation de la question 1.1

La question 1.1 a été résolue en construisant une surface continue de volatilité implicite à partir des données discrètes du fichier CSV. Les points de marché ont d'abord été chargés puis organisés sur une grille en maturité et en strike. À partir de cette grille, une surface $\sigma_{\text{imp}}(T, K)$ a ensuite été définie au moyen d'une interpolation bilinéaire.

Les tests réalisés ont confirmé que :

- la surface retrouve exactement les valeurs de la grille aux points connus ;
- les valeurs interpolées entre strikes sont cohérentes ;
- les valeurs interpolées entre maturités sont cohérentes ;
- les cas de bord simples sont correctement pris en charge.

La première étape du projet est donc validée.

4.2 Visualisation de la surface de volatilité implicite

La figure ci-dessous présente la surface de volatilité implicite obtenue à partir des données de marché et de l'interpolation bilinéaire.

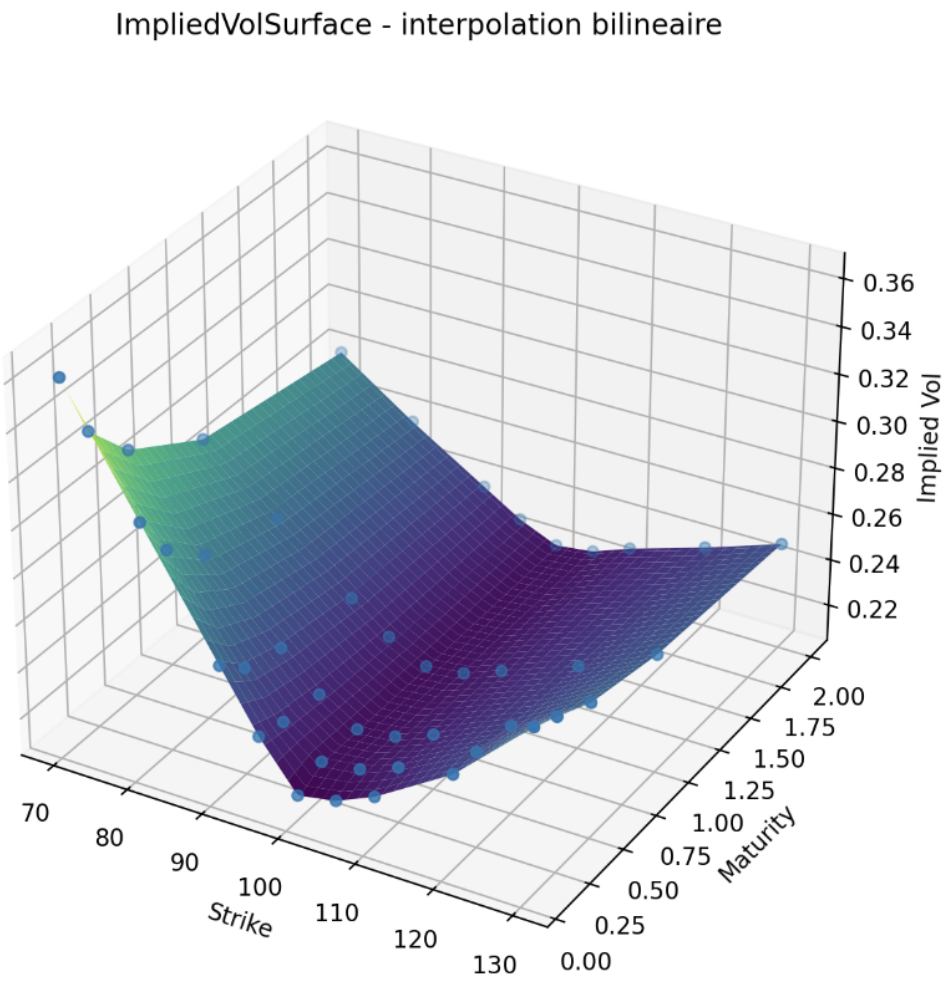

FIGURE 1 – Surface de volatilité implicite interpolée en fonction du strike et de la maturité.

La visualisation permet de vérifier qualitativement que la surface est régulière et cohérente avec les points de marché observés. Elle constitue également une étape utile avant le calcul des dérivées numériques nécessaires à la formule de Dupire.

4.3 Question 1.2 : volatilité locale

À partir de la surface implicite interpolée, une surface de volatilité locale a été construite en utilisant la variance totale implicite et les dérivées numériques en temps et en log-moneyness.

Cette étape est plus délicate numériquement que la simple interpolation de la surface implicite. Les premiers résultats montrent toutefois que l'utilisation de $w(y, T)$ améliore la stabilité du calcul et permet d'obtenir une surface locale exploitable, sous réserve de surveiller les zones de bord ou les zones où les dérivées sont plus sensibles.

4.4 Question 1.3 : tests d'arbitrage

Des tests numériques ont été mis en place afin de vérifier :

— la monotonie du prix en strike ;

— la convexité du prix en strike ;

— l'absence d'arbitrage calendaire.

Ces tests permettent de valider que la surface construite reste compatible avec des contraintes financières élémentaires.

4.5 Comparaison des schémas sous Heston

Dans l'option A, plusieurs schémas de simulation ont été comparés.

Le schéma d'Euler a servi de point de départ simple et lisible. Les premiers résultats obtenus montrent que le prix Monte Carlo issu de ce schéma reste compatible avec le prix semi-analytique de référence de Heston. L'écart observé demeure faible et peut être attribué à l'erreur statistique liée à Monte Carlo.

Le schéma TG fournit une alternative au schéma d'Euler, notamment pour mieux contrôler la positivité de la variance. Le schéma QE, plus ambitieux, cherche à mieux reproduire la structure conditionnelle de la variance. Enfin, l'implémentation de Broadie–Kaya réalisée dans ce projet joue le rôle d'une version structurée simplifiée, permettant d'élargir la comparaison numérique.

5 Analyse critique et limites

Le projet répond à l'essentiel des objectifs fixés, mais plusieurs limites doivent être mentionnées.

Premièrement, l'interpolation bilinéaire retenue pour la surface implicite est simple et robuste, mais elle n'est pas la méthode la plus sophistiquée possible. Une interpolation plus régulière pourrait améliorer la qualité des dérivées nécessaires à la volatilité locale.

Deuxièmement, le calcul de la volatilité locale par Dupire reste sensible au choix des pas numériques et à la qualité de la surface implicite. Les dérivées d'ordre un et deux peuvent être bruitées, en particulier aux bords du domaine.

Troisièmement, les tests d'arbitrage mis en place valident des propriétés importantes, mais ils ne constituent pas une preuve complète de robustesse. Une étude plus poussée exigerait davantage de cas numériques et une analyse plus systématique.

Quatrièmement, les schémas de l'option A ne sont pas tous au même degré de maturité :

— Euler est simple et sert de référence de base ;

— TG constitue un vrai schéma alternatif ;

— QE demande encore un raffinement pour être pleinement cohérent ;

— Broadie–Kaya est ici implémenté dans une version simplifiée.

Ainsi, le projet doit être compris comme une implémentation pédagogique sérieuse, mais non comme une librairie de production entièrement stabilisée.

6 Conclusion

Ce projet a permis d'implémenter en C++ une chaîne cohérente de traitement de modèles de volatilité, depuis les données implicites de marché jusqu'au pricing sous un modèle avancé.

La première partie a conduit à la construction d'une surface continue de volatilité implicite, puis à la mise en place d'une surface de volatilité locale via la formule de Dupire. Les tests effectués montrent que cette étape est globalement validée et fournit une base exploitable pour la suite.

La seconde partie, menée avec l'option A, a permis de mettre en œuvre plusieurs briques du modèle de Heston : génération de variables aléatoires, simulation jointe du spot et de la variance, pricing Monte Carlo et comparaison avec une référence semi-analytique.

Au total, ce projet articule de manière cohérente :

— des données de marché ;

— des fondements mathématiques ;
— des choix d'implémentation C++ ;
— des résultats numériques ;
— une analyse critique des limites.

Il constitue une base solide pour approfondir ensuite la qualité numérique de la calibration locale et le raffinement des schémas de simulation avancés.